Governance Drift: Measuring Divergence Between Approved Intent and Operational Reality in Cloud-Native Systems

Obede Bessa
Independent Researcher — obedebessa@gmail.com

***Abstract*—Configuration drift is the divergence between desired and observed configuration. We identify a broader relation that reconciliation does not decide and can even create: *governance drift*, the divergence between operational reality and the state, policy context, authorization basis, intent, evidence, or environment assumptions under which that reality was approved. A resource may match Git exactly while violating a superseding policy, relying on an expired exception, running an unapproved artifact, or reflecting an unauthorized rollback. We contribute (i) a taxonomy separating configuration, policy, authorization, intent, evidence, and environment drift, with governance consistency expressed componentwise; (ii) a model based on an approved-state snapshot, time-indexed governance contexts, and a governance-distance vector; (iii) counterexamples and a dependency result showing why configuration equality cannot decide governance consistency; (iv) a vendor-neutral, cumulative evidence-tier architecture; (v) an executed, reproducible closed-world study over nine scenarios, including evidence decay, with runtime and benign governance churn in the control; and (vi) a bounded execution on a pinned Kind/Flux/Kyverno stack, one run per scenario. In the modeled world, normalized configuration comparison detects one scenario of nine. Adding policy re-evaluation, authorization validity, artifact lineage, and environment inventory produces a strict detectability staircase, with each tier detecting the scenarios whose deciding facts it carries and no false alarms under modeled benign governance churn. An unnormalized comparator alarms on 71% of control events in our churn model. In the live laboratory, all nine injections began from a consistent baseline and produced the expected class-resolved verdict, with injection-to-verdict latencies of 0.137–8.571 seconds. This bounded execution demonstrates feasibility and occurrence on one real stack; it does not estimate prevalence, reliability, production effectiveness, or latency distributions.**

***Index Terms*—Governance drift, configuration drift, GitOps, policy as code, cloud governance, Kubernetes, continuous compliance, DevOps, distributed systems.**

I. Introduction

Reconciliation-based delivery gave cloud operations a powerful invariant: the running system converges to a declared desired state, and divergence—*configuration drift*—is detected and repaired continuously [1, 2, 3, 4, 5]. The invariant is so useful that it has quietly become a proxy for a much stronger claim. When the reconciler reports SYNCED and the drift detector reports clean, operators, auditors, and dashboards read: *the system is in the state we approved*. This paper is about the gap between those two statements.

Consider a payments service whose deployment was approved on March 3rd: the manifest matched revision `rev-42`, satisfied policy version π_7, carried a service-owner approval bound to image digest `sha256:aaa...`, and traced to ticket `TKT-99`. Months later the cluster still matches its Git manifest byte for byte. Meanwhile: the security team superseded π_7 with π_8, which the running configuration violates; an emergency exception granted during an incident expired after four hours and has now been silently in force for six weeks; the registry re-tagged `svc:1.4.2` so the pods run a digest no approval ever referenced; and a well-meaning rollback reverted Git to `rev-37`, content whose approval belongs to a different change. Configuration drift: zero. Every governance fact that made the deployment legitimate on March 3rd: gone. We call this condition *governance drift*: the operational state has diverged not from its manifest but from the state, policy context, authorization basis, and intent *under which it was approved*.

The distinction is not terminological. Configuration drift is a relation between two configurations—desired and observed—and is therefore decidable by comparison, which is exactly what reconcilers and drift tools [6, 7] implement. Governance drift is a relation between the present state and a *past approval event together with its evolving context*, and we establish the separation in two complementary forms: constructed, tool-realizable counterexamples, and a definitional-dependency result showing that no detector whose inputs are configuration pairs—however sophisticated its diffing—can decide governance consistency, because the deciding facts are not functions of those inputs (Proposition 1). Worse, reconciliation can *manufacture* governance drift while eliminating configuration drift: the rollback scenario above is converged by the reconciler into a state that no current approval covers (Section III).

Our executed study makes the separation concrete. We implement a governance-drift detector evaluated at cumulative evidence tiers—configuration comparison; plus policy re-evaluation; plus authorization records with validity semantics; plus artifact lineage; plus environment inventory against recorded assumptions—against nine injected drift scenarios (including evidence decay) and a no-drift control containing both runtime churn and *benign governance churn* (satisfied policy revisions, approved updates, hygienic exceptions), under a

counterfactual scoring rule that credits information rather than alarm volume. The result is a strict detectability staircase (Table II): normalized configuration comparison detects exactly one scenario of nine; each evidence tier detects precisely the scenarios whose deciding facts it carries; the governance tiers achieve zero false alarms *earned against* governance churn; and an unnormalized comparator alarms on 71% of benign control events while carrying no information about seven of the nine governance scenarios.

A. Research Questions

RQ: Can divergence between approved intent and operational reality be formally represented and measured as governance drift?

RQ1: Which forms of governance drift occur in cloud-native systems?

RQ2: Can governance drift be detected using existing control-plane evidence?

RQ3: How quickly does governance drift become visible?

RQ4: Which forms of drift are detectable through configuration comparison alone, and which require authorization or evidence context?

RQ1 is answered by the taxonomy and scenario catalog (Sections III and VI); RQ2 and RQ4 by the dependency result (Proposition 1), the tiered architecture (Section V), the executed detectability study (Section VI), and the bounded live laboratory (Section VII); RQ3 by the study's event-time measurements and the laboratory's injection-to-verdict wall-clock observations.

B. Contributions and Epistemic Status

C1. Taxonomy (conceptual): formal definitions separating configuration, policy, authorization, intent, evidence, and environment drift, with governance drift as their aggregate—and with the explicit commitment that governance drift must not, and provably does not, reduce to configuration drift (Section III).

C2. Formal model (conceptual/analytic): approved-state snapshot $G_{\text{app}}(t_0)$, observed state $G_{\text{obs}}(t)$, time-indexed governance contexts $\mathscr{P}(t), \mathscr{A}(t), \mathscr{I}(t)$, and a component-wise governance-distance vector $\mathsf{GD}(t)$, with a reasoned rejection of a primitive scalar distance (Section IV).

C3. Separation results (analytic): constructed counterexamples and a definitional-dependency proposition establishing that configuration equality does not imply governance consistency and that no configuration-pair detector decides it—deliberately elementary, and billed as such (Sections III and IV).

C4. Detection architecture (design): a vendor-neutral tiered architecture mapping each drift class to the minimal evidence tier that decides it (Section V).

C5. Executed detector study (empirical, closed-world): nine scenarios × six detector tiers × twenty seeds, with counterfactual scoring, class-set ground truth, and a control containing runtime *and* benign governance churn; all code and data released, matrix generated programmatically (Section VI). Its role is precisely scoped: it validates that the reference semantics realize the model's dependency structure and quantifies noise behavior; it is not field evidence, and it makes no claim about any real cluster.

C6. Bounded Kubernetes/GitOps laboratory (empirical, executed): a pinned Kind/Flux/Kyverno stack, local Git and OCI services, approved-state recorder, tiered evaluator, and executable injections for S1–S9. In one run per scenario, all nine baselines were consistent and all nine verdicts matched the expected class, with 0.137–8.571 s injection-to-verdict latency (Section VII). The run establishes feasibility and occurrence on this stack, not prevalence, reliability, or a latency distribution.

Established results are cited; analytic claims are proved in the model; the closed-world and live-stack empirical claims are separated explicitly; and the larger repeated-trial laboratory protocol remains a future extension.

C. Scope and Non-Goals

We address drift *after* an approved deployment; admission-time enforcement [8, 9, 10, 11] is complementary and assumed imperfect (break-glass paths exist by design). We propose no product; the detector logic we release is a reference semantics, not a system. And we take no position on *which* governance controls organizations should impose—the model makes drift relative to whatever was approved under whatever policy; the choice of policy is the organization's.

II. Background: The Reconciliation Blind Spot

A. Desired-State Delivery and Its Invariant

GitOps-style delivery declares desired state in a versioned repository and runs controllers that continuously converge the runtime toward it [1, 2, 3, 4]; operational practice treats the resulting convergence signal as a primary health indicator [12]. The reconciler's contract is a fixed-point property over *configurations*: $G_{\text{obs}} \to$ desired. Drift detection in current practice—reconciler status, cloud configuration recorders, IaC drift tools [6, 7], posture management [13]—implements variations of the same comparison, and configuration-management guidance institutionalizes it as the control objective [14, 15].

Three properties of this regime matter here. First, the comparison's *reference* is the current desired state, not the approved state: if the desired state itself changes illegitimately, the reconciler converges to the illegitimate state and reports health. Second, the comparison's *vocabulary* is configuration: facts that are not configuration—policy versions, approval windows, artifact lineage—are outside its type. Third, the comparison is *memoryless*: it relates two present states and consults no past event, whereas approval is precisely a past event.

B. The Governance Context Moves

Meanwhile, the context that made a deployment legitimate evolves on its own clocks. Policy-as-code is versioned and revised continuously [9, 10]; organizations supersede controls without redeploying workloads, under regulatory regimes that presume change remains authorized over time [16, 15]. Authorizations are temporal by nature—emergency exceptions with windows, approvals with scopes, roles that are granted and revoked [17, 18]; their validity changes with no configuration event at all. Artifact references are mutable (tags move; registries re-tag), so the binding between an approval's subject and the running binary can rot while every manifest byte stays fixed [19, 20]. And parts of the environment that govern a workload's risk (IAM scope, network exposure, cloud-resource settings) are frequently *unmanaged*: no desired state exists for them, so no comparison can flag their change.

C. Why This Produces a Blind Spot, Not a Bug

None of the mechanisms above is broken. Each does exactly what it is specified to do; the blind spot is an emergent property of their composition: the only continuously-checked invariant in the system is configuration equality, while the property stakeholders actually care about—*this running state is one that current policy, valid authorization, and recorded intent still cover*—is checked jointly exactly once, at admission, and thereafter only in fragments: policy engines background-scan existing resources against *current* policy [10, 9], configuration recorders track unmanaged resources against their own baselines [6], and continuous-validation features re-check artifact policy [11]—each against its own reference, none against the admitted basis, and no component checking validity clocks on the authorization itself. The general property—joint consistency of running state with current context *and* the admitted basis, with class-resolved verdicts—has, to our knowledge, neither a name, a model, nor a detector semantics, although each fragment above supplies detection machinery one tier of it can reuse (Section V). Naming and modeling it is this paper's job; Section III begins with the vocabulary.

A terminological note to prevent collisions: "drift" is also used for statistical distribution change in learning systems (concept drift [21]); our subject is unrelated. Industry writing on "cloud governance and drift" uses governance as the *program* that manages configuration drift [22]; our subject is drift *of governance itself*—a relation the configuration vocabulary cannot express, as Section III now makes precise.

III. A Taxonomy of Drift (RQ1)

We fix vocabulary relative to an *approval event*: the moment a change was legitimized for an environment. Figure 1 shows the two planes that subsequently evolve. All definitions are made precise in the model of Section IV; here we give their content and their separations.

Definition 1 (Approved State)*: The* approved state $G_{\text{app}}(t_0)$ *of a deployment is the snapshot, at approval time* t_0*, of: the approved configuration (manifest content and the artifact digests it resolves to), the policy version under which it was evaluated, the authorization basis (approvals, exceptions, their subjects, scopes, and windows), the intent linkage (the change record and revision the approval covers), and the declared environment assumptions within which the risk assessment was made.*

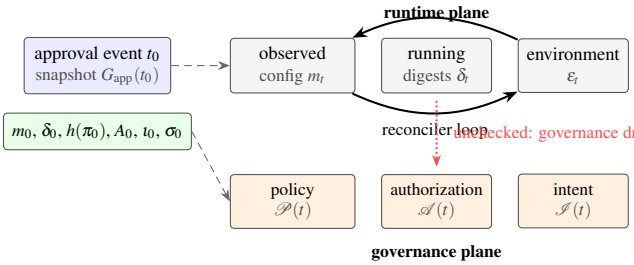

Fig. 1. Approved intent versus runtime reality. The approval event at t_0 freezes a snapshot: configuration, policy version, authorization basis, intent linkage, and environment assumptions. Afterward, two planes evolve independently: the runtime plane (observed configuration, running artifacts, environment) and the governance plane (policy versions, authorization validity, intent records). Reconcilers continuously compare within the runtime plane (solid loop); nothing in standard practice compares the runtime plane against the *evolved* governance plane (dashed gap)—the space in which governance drift lives.

Definition 2 (Configuration Drift)*: Divergence between the* current desired *configuration and the* observed *configuration:* $G_{\text{obs}}(t) \neq desired(t)$*. This is the classical notion [6, 7, 14], decidable by comparison of present states.*

Definition 3 (Policy Drift)*: Divergence between the policy version pinned in* $G_{\text{app}}(t_0)$ *and the policy context now governing the environment,* as it bears on this deployment*:* $\mathscr{P}(t)$ *has changed such that the running configuration would not be admitted under it today, although it was compliant when approved.*

Definition 4 (Authorization Drift)*: Invalidation or mismatch of the authorization basis without any configuration event: an approval's window lapsed while its effect persists (expired emergency exception); an approval was revoked; or the running artifact's digest is not the subject any valid approval refers to.*

Definition 5 (Intent Drift)*: Divergence between the content now driving the runtime and the intent record the authorization covers: the desired state was changed (e.g., rolled back) to content whose approval basis is absent or belongs to a different change, and reconciliation faithfully executes it.*

Definition 6 (Evidence Drift)*: Decay of the records connecting the running state to its approval basis: the chain (revision* $\rightarrow$ *artifact* $\rightarrow$ *deployment* $\rightarrow$ *approval) can no longer be established from surviving evidence, so governance consistency becomes* undecidable *rather than false. Evidence drift converts governance questions from answerable to unanswerable and is therefore tracked as its own class rather than folded into the others.*

Definition 7 (Environment Drift)*: Change, outside the man-*

aged configuration's scope, of environment properties that the approval's risk assessment assumed: IAM scope broadened, network exposure altered, an adjacent cloud resource modified out of band—no desired state exists for the changed property, so no configuration comparison can even represent the divergence.

Definition 8 (Governance Drift)*: The condition in which the operational state of an environment diverges from the state, policy context, authorization basis, intent, or environment assumptions under which it was approved: the componentwise condition over Definitions 3 to 7, together with those configuration divergences (Definition 2) that bear on approved properties. A deployment is* governance-consistent *at t when every component distance of Section IV is zero and none is undecidable. The components are not mutually exclusive: one incident may drift several at once (an unapproved rollback drifts intent* and *leaves the runtime on digests no valid approval covers), which is one reason the model of Section IV keeps them as a vector.*

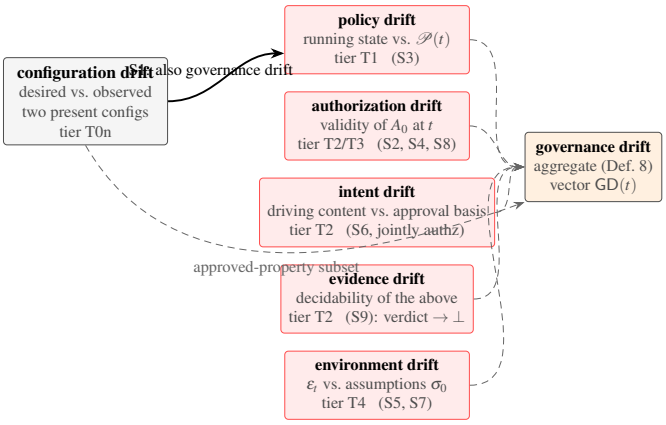

Fig. 2. The drift taxonomy. Configuration drift (left) is a relation between two present configurations. The governance-drift classes (right) each relate the present state to a component of the approved snapshot or its evolved context; their aggregate is governance drift (Definition 8). Annotated under each class: the evidence tier of Section V that first decides it, as measured in Section VI.

A. Risk Channels and Severity

The six components are equivalent only as coordinates of the governance distance vector; they are *not* equivalent in consequence. A raw count therefore cannot stand in for severity. Each class opens a different primary risk channel and implies a different default disposition, as summarized in Table I. Configuration drift first threatens operational integrity; policy drift threatens compliance with the currently governing control; authorization drift threatens security legitimacy; intent drift threatens the legitimacy of the change itself; evidence drift threatens audit and forensic decidability; and environment drift changes the exposure or assumption boundary around the approved system.

This mapping is causal, not ordinal: drift class $\to$ risk channel $\to$ response family. Within a class, severity must still be conditioned on asset criticality, affected subjects, exposure time, reversibility, and the confidence permitted by surviving evidence. Across classes, a low-impact authorization mismatch can be less severe than a configuration fault that stops a safety-critical service. The taxonomy thus supports class-specific prioritization without pretending that one universal weight orders every incident.

B. The Central Separation

The taxonomy's load-bearing claim is that Definition 8 does not reduce to Definition 2. We establish it constructively and then in general form.

1) Constructed counterexamples

Figure 3 presents the canonical scenario in full: a deployment whose observed configuration equals its desired configuration at every instant shown, and which passes from governance-consistent to governance-inconsistent three separate times—at the policy supersession (policy drift), at the exception expiry (authorization drift), and at the registry re-tag (authorization drift at digest granularity). In the fourth panel the desired state itself is rolled back and the reconciler *eliminates* the transient configuration drift by converging to content with no valid approval basis: reconciliation manufactures intent drift while restoring configuration consistency. Each panel is realizable with standard tooling, and each is one of the executed scenarios of Section VI.

2) The dependency form

Proposition 1 (Configuration equality does not decide governance consistency)*: Let D be any detector whose inputs are configuration pairs* $(desired(t), G_{\mathrm{obs}}(t))$*—including their full histories. There exist worlds* w_1, w_2 *with identical configuration-pair histories in which the same deployment is governance-consistent in* w_1 *and governance-inconsistent in* w_2*. Hence no such D decides governance consistency, for any diffing sophistication.*

Proof. Take w_1 and w_2 to agree on all configurations at all times and differ only in the governance plane: in w_2 the policy version governing the environment was superseded after t_0 by one the running configuration violates (or: an exception's window elapsed; or: a valid approval in w_1 is absent in w_2 for the same content). These differences live in $\mathscr{P}(t), \mathscr{A}(t), \mathscr{I}(t)$, which are not functions of the configuration pair; both worlds present D with identical inputs, and Definition 8 assigns them different values. D outputs the same answer in both, so it errs in one. □

The proposition is elementary—a definitional-dependency observation, and we bill it as such rather than as a deep impossibility theorem. Its force is architectural: it says deployed drift detectors are not approximations of governance-drift detectors but detectors *of a different relation*, and no incremental improvement of configuration diffing closes the gap *as a decision procedure*. Two bounds on what it claims: it rules out *deciding*, not statistical *hinting*—in real estates, governance-plane events often correlate with configuration-plane events (policy

TABLE I
CLASS-RESOLVED RISK CHANNELS AND DEFAULT RESPONSE FAMILIES. THESE ROWS ARE NOT A SCALAR RANKING: ACTUAL SEVERITY DEPENDS ON BLAST RADIUS, EXPOSURE DURATION, REVERSIBILITY, CONSEQUENCE, AND EVIDENCE COVERAGE.

Drift class	Primary risk channel	Governance question	Default response family
Configuration	Operational availability and integrity	Does observed state still match the managed desired state?	Reconcile, contain, or accept an explicit runtime exception
Policy	Compliance and control validity	Would the running state satisfy the policy governing it now?	Re-evaluate, migrate under an SLA, or document a current exception
Authorization	Security and privilege legitimacy	Is the running subject still covered by valid authority?	Revoke, block, contain, or page the accountable owner
Intent	Governance and change legitimacy	Does the deployed revision implement the change that was approved?	Halt propagation, restore the authorized revision, or obtain new approval
Evidence	Audit and forensic decidability	Can the approval basis and lineage still be established?	Preserve records, escalate, and report *undecidable* rather than compliant
Environment	Assumption and exposure boundary	Do unmanaged surroundings still satisfy the recorded risk assumptions?	Contain exposure and reassess the approval in the changed environment

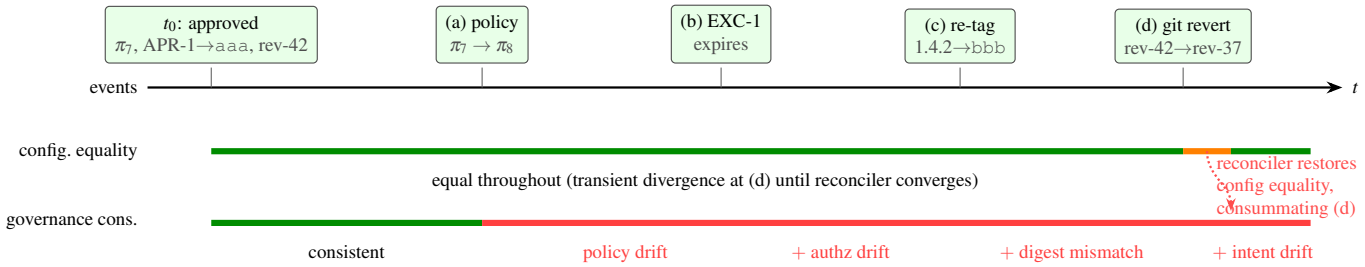

Fig. 3. Configuration-consistent but governance-inconsistent, in four panels. Timeline of one deployment: observed configuration (bottom band) remains equal to desired configuration throughout—configuration drift is zero at every instant after reconciliation—while governance consistency (top band) is broken successively by (a) policy supersession $\pi_7 \to \pi_8$, (b) expiry of emergency exception EXC-1 whose effect persists, (c) registry re-tag moving `svc:1.4.2` to a digest no approval references, and (d) a Git rollback that the reconciler faithfully converges to, eliminating transient configuration drift while stranding the runtime on content whose approval belongs to a different change. Panels correspond to Scenarios S3, S2, S4, and S6 of Section VI.

revisions ship alongside config pushes; rollbacks are visible in Git history), so configuration histories may carry probabilistic signal about governance state; how much is an empirical question for the laboratory, which our closed world deliberately does not answer. And it says nothing against comparison as a *mechanism*: the governance tiers of Section V are themselves comparisons—against recorded bases, validity clocks, and coverage sets—which is precisely the point: the input vocabulary, not the mechanism, is what must widen.

3) What configuration drift remains

The separation does not demote configuration drift: unauthorized manual change (Section VI, S1) is both configuration and governance drift, and configuration comparison is its right detector. The taxonomy's point is containment, not replacement: configuration drift is one class among six, the only one decidable in the configuration vocabulary—which the study confirms empirically as a detectability staircase whose first step is exactly that one class.

IV. A FORMAL MODEL OF GOVERNANCE DRIFT (RQ2, RQ4)

A. States and Contexts

Fix a deployment x into environment e, approved at t_0.

Approved snapshot. Per Definition 1,

$$G_{\text{app}}(t_0) = \big(m_0,\ \delta_0,\ h(\pi_0),\ A_0,\ \iota_0,\ \sigma_0\big)\ :$$

the approved manifest content m_0 and the artifact digests δ_0 it resolved to at approval; the pinned policy version; the authorization basis A_0 (a set of approval and exception records, each with subject digest, scope, window, and revocation status); the intent linkage ι_0 (change record, revision); and the environment assumptions σ_0 (the declared properties of e—identity scopes, exposure—under which risk was assessed). $G_{\text{app}}(t_0)$ is immutable by construction: it is a record of an event.

Observed state. $G_{\text{obs}}(t) = (m_t, \delta_t, \varepsilon_t)$: the observed configuration, the digests actually executing, and the observed environment properties corresponding to σ_0.

Contexts. Three time-indexed context streams: $\mathscr{P}(t)$, the policy version(s) governing e at t with their evaluation semantics; $\mathscr{A}(t)$, the authorization records valid at t (windows elapsed, revocations applied); and $\mathscr{I}(t)$, the intent records and the desired-state revision currently driving delivery. Evidence availability, where needed, is a fourth stream $\mathscr{E}(t)$ (the records from which the above can be established).

B. The Governance-Distance Vector

We define one distance per taxonomy class; each is a function of exactly the inputs its class needs, which is what makes the detectability results of Section VI possible to state.

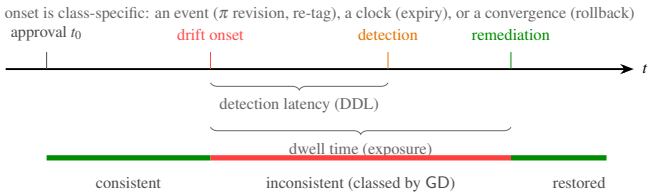

Fig. 4. The drift lifecycle and its temporal quantities (Definition 9). Onset is class-specific—an organizational event (policy revision), a clock crossing (exception expiry), an exogenous event (registry re-tag), or a reconciler convergence (rollback). Detection latency is the tier-dependent quantity measured in Section VI; dwell time is the exposure window that remediation closes, with the remediation family determined by the drifted component (Section IV-C).

- $d_{\text{conf}}(t)$: a normalized difference over the managed configuration graph between desired(t) and m_t (field-level graph diff with runtime-owned fields factored out; Section VI shows the normalization is not cosmetic but the difference between signal and noise).
- $d_{\text{pol}}(t)$: the set of controls in $\mathscr{P}(t)$ that the running state violates although G_{app} was compliant at t_0—empty iff admission would re-succeed under today's policy. Reported as a violated-control set, not a number: policy versions form a revision structure, not a metric space, and "how far" is policy-weighted judgment, not geometry.
- $d_{\text{auth}}(t)$: the validity gap of the authorization basis: expired-but-effective exceptions, revoked approvals still relied on, and the digest mismatch $\delta_t \not\subseteq \text{subjects}(\mathscr{A}(t))$.
- $d_{\text{int}}(t)$: the mismatch between the revision driving delivery and the revision the valid authorization covers ($\mathscr{I}(t)$ versus ι_0 and its legitimate successors).
- $d_{\text{env}}(t)$: divergence $\varepsilon_t \neq \sigma_0$ on the assumed environment properties.
- $d_{\text{evd}}(t)$: the undecidability gap—components of the above that cannot be evaluated from $\mathscr{E}(t)$; a three-valued semantics (consistent / inconsistent / undecidable-at-ℓ) rather than a magnitude.

Definition 9 (Governance Distance; Consistency)*:* $\mathsf{GD}(t) = \big(d_{\text{conf}}, d_{\text{pol}}, d_{\text{auth}}, d_{\text{int}}, d_{\text{env}}, d_{\text{evd}}\big)(t)$*, and* $\mathsf{Consistent}(x,t)$ *holds iff every component is empty/zero and none is undecidable. Onset of governance drift is* $\inf\{t > t_0 : \neg\mathsf{Consistent}(x,t)\}$*;* dwell time *is the duration of inconsistency;* detection latency *is detection time minus onset.*

C. Why the Distance Is a Vector, Not a Scalar

A scalar GD was the natural first proposal, and we reject it as primitive for three reasons. *Incommensurability*: the components take values in different structures (graph diffs, control sets, validity states, three-valued verdicts); any scalarization imports policy weights, and those weights are governance choices, not properties of the system. *Remediation asymmetry*: the components have different remediations—reconcile (d_{conf}), re-evaluate and re-approve or migrate (d_{pol}), re-authorize or terminate the exception (d_{auth}), restore or re-approve intent (d_{int}), restore assumptions or re-assess (d_{env}), repair evidence (d_{evd})—so collapsing them destroys exactly the information an operator needs. *Temporal structure*: components drift on different clocks (policy revisions are discrete organizational events; exception expiry is a scheduled instant; registry re-tags are exogenous), so a scalar time series conflates processes that must be attributed separately. A weighted composite for reporting is legitimate *on top of* the vector (Section VIII), with published weights, exactly as a risk score summarizes but does not replace findings.

D. Detectability, Formally

Each component distance is a function of specific streams: d_{conf} of (desired, G_{obs}); d_{pol} additionally of $\mathscr{P}(t)$ and G_{app}'s pin; d_{auth} of $\mathscr{A}(t)$, G_{app}, and δ_t; d_{int} of $\mathscr{I}(t)$ and G_{app}; d_{env} of ε_t and σ_0; d_{evd} of $\mathscr{E}(t)$. Proposition 1 is the negative direction of this dependency structure; the positive direction is equally direct:

Proposition 2 (Tier sufficiency)*: A detector with access to the streams a component depends on decides that component's consistency (in the closed-world model, exactly; in the field, up to the fidelity of the streams). Consequently the minimal detector for full governance consistency is one that joins the configuration comparison with the policy, authorization, intent/lineage, and environment streams—the tiered architecture of Section V, whose scenario-to-tier realization Section VI checks cell by cell.*

Proof. Each component distance in Section IV-B is defined as a computable function of the named streams; providing the streams provides the function's inputs. □

The proposition is an input-enumeration result; whether a concrete implementation realizes it is a separate, checkable question—Section VI performs exactly that check on our released reference semantics, and *only* that check: agreement there validates the implementation, not the model.

Two modeling notes bound the claims. First, *granularity*: whether the registry re-tag scenario is "configuration" drift depends on reference semantics—under tag semantics the configuration is unchanged; under digest semantics it changed. The model takes no side: it requires only that G_{app} record resolved digests, which relocates the question from metaphysics to evidence (what did the approval bind?), where Section VI shows it is decidable at the lineage tier. Second, *legitimate context change*: policy supersession does not make drift *culpable*—the deployment was legitimate under π_7 and the organization changed the rules. The model deliberately measures divergence, not blame; whether policy drift triggers migration, exemption, or re-approval is a governance decision the vector informs (Section X).

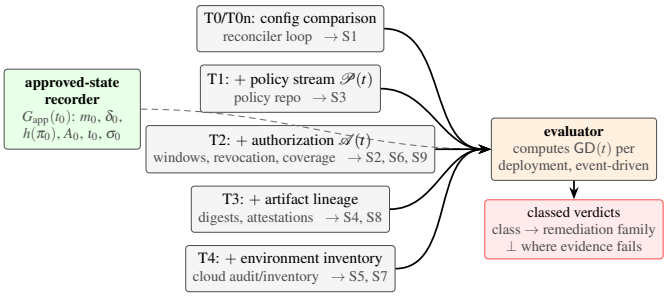

Fig. 5. Tiered governance-drift detection. The approved-state snapshot (left) is recorded at admission. Evidence tiers (center) cumulatively widen the detector's vocabulary: T0/T0n configuration comparison (the reconciler's existing loop, naive or churn-normalized), T1 policy-version stream, T2 authorization records with validity semantics, T3 artifact lineage, T4 environment inventory. The evaluator (right) recomputes the component distances of Section IV-B continuously and emits classed drift verdicts with the remediation family each class implies. Annotations show which of Scenarios S1–S8 each tier first detects, as measured in Section VI.

V. A Tiered Detection Architecture

Propositions 1 and 2 together prescribe the architecture: governance-drift detection is configuration comparison *plus* the minimal additional evidence streams, joined against the approved snapshot. Figure 5 organizes it as cumulative tiers, each named for what it adds and each mapped to the drift classes it first decides.

A. Components

Approved-state recorder. At admission, snapshot $G_{\text{app}}(t_0)$—manifest content, *resolved* digests, policy-version pin, authorization records with windows, intent linkage, environment assumptions—and persist it durably and independently of the delivery platforms. This is the architecture's one hard prerequisite: without a recorded approval basis there is nothing to drift *from*; where such records are absent or decayed, drift verdicts degrade to undecidable (d_{evd}), which the evaluator must report as such rather than as consistency.

Evidence tiers. *T0/T0n*: the desired/observed comparison every reconciler already performs [3, 4, 6]; T0n normalizes runtime-owned fields (replica counts under autoscaling, rollout hashes)—the study quantifies why the naive form is unusable as a governance signal. *T1*: the policy-version stream from the policy engine's repository [9, 10]; the evaluator re-evaluates the running state against $\mathscr{P}(t)$ and compares with the pinned basis. *T2*: authorization records with validity semantics—expiry, scope, revocation—so that exceptions and approvals are objects with clocks, not booleans; ITIL-style change records [18] and platform approvals both map here. *T3*: artifact lineage—digest-level linkage from running workloads to approval subjects, the slice pioneered by supply-chain attestation [19, 20, 23, 11]. *T4*: environment inventory—observation of the unmanaged properties assumed at approval (IAM scope, exposure) and comparison against the *recorded* assumptions σ_0 in G_{app}, from cloud inventories, audit streams, and telemetry correlation [24, 13, 25]. Mechanically this is baseline comparison—CSPM's move—and we say so plainly; the difference is the reference (the approval event's recorded assumptions rather than a benchmark) and the joined, class-resolved verdict.

Evaluator. A continuous process (or scheduled sweep) computing $\mathsf{GD}(t)$ per deployment: cheap incremental checks on context-change events (policy revision published, exception clock elapsed, registry event) rather than full re-scans—governance drift, unlike configuration drift, is mostly *event-driven* on the governance plane, which is why several scenarios in Section VI are detectable with zero latency at the tier that carries the relevant event.

B. Relation to Existing Mechanisms

The architecture is deliberately assembled from streams that already exist in mature platforms; its novelty is the *join semantics* against a recorded approval basis and the class-resolved verdict. Admission control [8, 9, 10] evaluates the same predicates once, at entry; continuous-validation features [11] re-evaluate the artifact-policy slice; posture management [13] scans environment configuration against benchmark policy without any approved-state reference. Indeed, per-tier continuous detection largely *exists* in cited tooling, and the architecture credits it explicitly: Kyverno's background scanning re-evaluates existing resources against current policy (our T1) [10]; configuration recorders track unmanaged resources (much of T4) [6, 13]; continuous validation re-checks artifact policy (part of T3) [11]. What no deployed mechanism provides, to our knowledge, is the join: all tiers evaluated against the *admitted basis* G_{app}, with validity clocks on the authorization itself, and verdicts that distinguish "violates policy" from "was never approved" from "was admitted legitimately and the world moved"—the distinctions that determine remediation (Section IV-C). The contribution is the relation and its join semantics, not the comparison machinery, and Section IX restates novelty in exactly those terms.

VI. Executed Study: Detectability Across Evidence Tiers (RQ2–RQ4)

We now measure the taxonomy's detectability structure. The study is executed and fully reproducible (≈340 lines of dependency-free Python, released with all outputs; the results table, including its header, is generated programmatically by the same run). Its epistemic scope is stated plainly: it is a *closed-world evaluation of detector semantics*—the world, its scenarios, and the detectors are all models—so it establishes what each evidence tier can and cannot decide *in principle*, with event-time latencies. It does not measure any real cluster; wall-clock behavior on real infrastructure is the laboratory's job (Section VII).

A. Modeled World and Scenarios

The world models one deployment with the approved snapshot of Section IV-A—including recorded environment assumptions σ_0—and streams for desired state (Git), observed cluster state, registry contents, policy versions *with evaluable requirement sets* (so that T1 implements Definition 3's admission

re-evaluation, not a version-string comparison), authorization records as a set with subject digests, covered revisions, windows, and revocation (so that legitimate successors are modeled), and two unmanaged environment properties. Each run is 400 events; drift is injected at event 150. The nine scenarios follow the taxonomy and the constructed examples of Figure 3:

S1 manual in-cluster change (observed field diverges from desired);

S2 expired emergency authorization (exception granted with a 40-event window; nothing removes it at expiry);

S3 policy version change post-deploy ($\pi_7 \to \pi_8$, which the running configuration violates);

S4 artifact substitution (same tag re-pointed to a new digest, rolled onto nodes; manifests unchanged);

S5 IAM permission expansion out of band (unmanaged);

S6 Git rollback without new approval (reconciler converges to the reverted content; approval binds the newer revision);

S7 out-of-band cloud console modification (unmanaged);

S8 approval mismatch (deployed digest is not any approval's subject);

S9 approval-record deletion: the records establishing the basis are lost (evidence drift—Definition 6—exercised directly);

plus **S0**, a no-drift control. Churn is of two kinds, both present in every stream including the control: *runtime churn* (30% of events: autoscaler replica changes, rolling-restart hash changes) and *benign governance churn* (2% of events: policy revisions the running state satisfies, approved updates that legitimately advance revision, digest, and approval together, and hygienic exceptions removed at expiry). The governance churn exists so that the governance tiers' false-alarm rates are earned against realistic governance noise rather than measured against a silent plane.

B. Detectors and Counterfactual Scoring

Six detector tiers implement Section V: T0 (naive configuration comparison, including runtime-owned fields), T0n (normalized), and cumulative T1–T4, each a pure function of its tier's visible streams, so tier capabilities are enforced by construction. Ground truth is a *class set* per scenario (components are a vector and may drift together, Definition 8: S6 drifts both intent and authorization); the detector reports the first nonzero component under a declared evaluation order, and classification is correct iff the reported class is in the set. Missing approval records produce the explicit undecidable/evidence verdict, never a polar one.

Scoring required care, and the naive arm is why. A detector that alarms constantly "detects" every scenario by coincidence; volume is not information. We therefore score *counterfactually*: for each (scenario, tier, seed), the detector runs on the drift stream and on a paired, same-seed, no-drift control stream with identical churn; the drift is *detected* iff the two alarm sequences differ, detection time is the first differing event, and classification is the class emitted there. Under this rule a constant alarmer carries zero information and scores zero. False alarms are counted on the control stream. Twenty seeds per cell; onset for S2 is the expiry instant (the drift is the exception *outliving* its window, not its grant).

TABLE II
DETECTABILITY MATRIX, GENERATED PROGRAMMATICALLY FROM THE RUN OUTPUTS (20 SEEDS PER CELL). ✓ = DETECTED AND CORRECTLY CLASSIFIED IN ALL SEEDS (SUPERSCRIPT: MEAN DETECTION LATENCY IN EVENTS WHEN >0); × = NOT DETECTED (NO INFORMATION UNDER COUNTERFACTUAL SCORING). FP = FALSE ALARMS PER 400-EVENT CONTROL RUN.

Scenario	**T0** naive cfg	**T0n** norm. cfg	**T1** +policy	**T2** +authz	**T3** +lineage	**T4** +env
S0 control (churn only)	FP: 285.4	FP: 0.0	FP: 0.0	FP: 0.0	FP: 0.0	FP: 0.0
S1	✓$^{+22.6}$	✓	✓	✓	✓	✓
S2	×	×	×	✓	✓	✓
S3	×	×	✓	✓	✓	✓
S4	×	×	×	×	✓	✓
S5	×	×	×	×	×	✓
S6	×	×	×	✓	✓	✓
S7	×	×	×	×	×	✓
S8	×	×	×	×	✓	✓
S9	×	×	×	✓	✓	✓

S1 manual change; S2 expired exception; S3 policy supersession; S4 artifact substitution; S5 IAM expansion; S6 unapproved rollback; S7 out-of-band cloud change; S8 approval mismatch; S9 approval-record deletion (evidence). Tiers are cumulative; T0n normalizes runtime-owned fields; latencies use expiry-inclusive semantics.

C. Results

Table II is the study's central artifact. Four findings, stated in the register the design supports.

Finding F1 (The staircase: implementation validation of tier dependencies)*: Detection forms a strict staircase realizing Proposition 2 cell for cell: T0n detects S1 and nothing else—one scenario of nine; T1 adds exactly the policy-drift scenario (S3), and—because it re-evaluates policy rather than comparing versions—raises no alarm on the satisfied policy revisions in the churn; T2 adds the validity- and coverage-dependent scenarios (S2, S6) and the evidence-decay scenario (S9, reported as the undecidable verdict, not a polar one); T3 adds the digest-level scenarios (S4, S8); T4 adds the unmanaged-environment scenarios (S5, S7). Every detection is correctly classified against the class-set ground truth in every seed. We state this finding's epistemic value precisely: the world and detectors are co-designed, so the exactness is expected, and the staircase is* implementation validation *of the model's dependency structure plus a noise-behavior measurement—not field evidence for the taxonomy, a role we do not claim for it. Its concrete content for RQ4 stands: in the reference semantics, one class is decidable by configuration comparison and five are not, and all six taxonomy classes (including evidence drift) are exercised.*

Finding F2 (Unnormalized comparison is noise; governance-tier quiet is earned)*: The unnormalized comparator T0 alarms on 285.4 of 400 control events (71.4% in our churn*

model)—and the rate is nearly flat across runtime churn rates of 10–50% (287, 285, 300 alarms per run), because alarms track the dwell *of divergent runtime-owned state, not the event rate. Under counterfactual scoring T0 still detects S1, but with mean latency 22.6 events versus 0 for T0n: its information is buried in its own noise. For the other eight scenarios it carries zero information while alarming continuously. We deliberately do not call T0 "the industry default"—production reconcilers normalize—but it bounds what comparison-without-normalization semantics yields, and it demonstrates the scoring rule doing its job. Conversely, the governance tiers' zero false alarms are now* earned*: the control contains satisfied policy revisions (which a version-comparison T1 would have flagged), legitimate approved updates (which a pinned-revision intent check would have flagged), and hygienic exceptions—and the definition-faithful detectors correctly ignore all of them.*

Finding F3 (Governance drift is event-detectable, with one designed exception)*: At the correct tier, every scenario is detected with latency 0 events (including S2 under expiry-inclusive semantics) and none requires scanning: the deciding facts arrive as events on governance streams (a policy revision, a clock crossing, a registry event, an inventory delta). RQ3's answer in the closed world is therefore: visibility is bounded by evaluation cadence on the governance plane, not by any intrinsic latency of the phenomenon—which sharpens what the laboratory must measure (wall-clock cadence effects, Section VII).*

Finding F4 (Reconciliation health is compatible with total governance failure)*: In seven of nine scenarios (all but S1 and transiently S6), configuration comparison at any sophistication reports clean throughout—while the deployment runs under superseded policy, expired authorization, unapproved content, or altered environment. This is Proposition 1 made operational, and it quantifies the blind spot claimed in Section II: a* SYNCED *reconciler is evidence about configuration, and about configuration only.*

D. Threats to Validity

Construct: detectors and world share a modeling vocabulary; the tiers' *exact* 0/1 staircase is a property of the closed world, where streams are noiseless and semantics are implemented as defined. Real streams are late, lossy, and ambiguous; the staircase's *shape* (which class needs which evidence) is the transferable claim—it follows from input dependencies (Proposition 2)—while the crispness is not. *Internal*: single-deployment world; one injection per run; no compound scenarios (the artifact supports composition, left as an extension). The declared evaluation order over components is one policy among several; because scoring uses class sets, reordering changes which member of a multi-component drift is reported, not correctness. Scenario S6's transient configuration signal (before the reconciler converges) is not credited to T0n because the comparison evaluates post-convergence state in our event ordering; on real clusters a fast comparator might glimpse it—a cadence question for the lab. *External*: the bounded laboratory (Section VII) establishes that the nine classes can be induced and correctly distinguished through real Kubernetes/GitOps interfaces on one pinned stack. It does not transfer the closed-world rates quantitatively or estimate field prevalence, reliability, stream loss, or performance under load; the repeated, churn-controlled protocol and its refutation conditions remain necessary for those questions.

VII. BOUNDED KUBERNETES/GITOPS LABORATORY

The closed-world study fixes semantics; a live stack tests whether the defined deficit classes can be induced and distinguished through real control-plane interfaces. We therefore executed a bounded laboratory over all nine scenarios. Its epistemic purpose is deliberately narrow: it is a feasibility and occurrence demonstration on one reproducible stack, not a field-prevalence study, reliability estimate, production benchmark, or latency distribution.

A. Build and Reproducibility Boundary

A single-node Kind v0.32.0 cluster running Kubernetes v1.36.1 [26] hosts Flux v2.9.4 [4] and Kyverno v1.18.2 [10]. A local smart-HTTP Git remote drives Flux; a local OCI registry enables tag/digest substitution; and a versioned inventory file represents environment properties outside the managed manifest. Bootstrap pins release manifests by SHA-256, preloads the required arm64 images, and records platform versions. An approved-state recorder snapshots G_{app}—manifest, resolved pod-image digest, policy pin, approval JSON with validity window, Git revision, and environment assumptions. A dependency-free evaluator reads the live Kubernetes API, Git, Kyverno PolicyReports, approval records, pod image IDs, and the inventory, with tier-restricted entry points T0n–T4. The complete bootstrap, injections, evaluator, raw observations, and generated table ship under `lab/`.

B. Bounded Protocol

The harness first restores the approved Git revision, policy π_7, approval record, artifact tag, deployment, and environment inventory. It requires the highest-tier evaluator to return *consistent* before each injection. It then executes S1–S9 once each, polls every 0.5 s until the minimum deciding tier returns a non-consistent verdict, and records expected class, observed verdict and class, and elapsed wall-clock time. Latency is measured from injection initiation to the first tier-appropriate verdict; for S2 it begins at exception expiry. Thus S4 includes registry mutation and rollout, S6 includes Git commit and Flux convergence, and S3 includes Kyverno's background-policy evaluation. The harness resets and verifies the baseline after every scenario. This is $n = 1$ per scenario: the values below are observations, not estimates of a distribution.

C. Observations

Table III reports all runs. Every scenario began from a governance-consistent baseline, and all nine outcomes matched

TABLE III
OBSERVED RESULTS FROM THE BOUNDED KIND/FLUX/KYVERNO EXECUTION. LATENCY IS INJECTION-TO-VERDICT WALL-CLOCK TIME IN SECONDS; S2 STARTS AT EXPIRY. "U" DENOTES AN EXPLICIT UNDECIDABLE VERDICT. ONE EXECUTION PER SCENARIO; NO RELIABILITY OR PREVALENCE INFERENCE IS MADE.

Scenario	Injection	Expected	Observed	Latency (s)	Correct?
S1	manual in-cluster change	configuration	configuration	0.14	Yes
S2	expired exception	authorization	authorization	0.18	Yes
S3	policy supersession	policy	policy	8.57	Yes
S4	artifact substitution	authorization	authorization	3.54	Yes
S5	IAM expansion	environment	environment	0.23	Yes
S6	unapproved Git rollback	intent	intent	6.92	Yes
S7	out-of-band LB change	environment	environment	0.27	Yes
S8	approval subject mismatch	authorization	authorization	0.19	Yes
S9	approval-record deletion	evidence	evidence (U)	0.15	Yes

the expected class (9/9). S9 correctly returned *undecidable/evidence*, rather than misstating missing evidence as compliance or violation. Observed latencies ranged from 0.137 to 8.571 s. The policy-supersession path was slowest (8.571 s), followed by the Flux-mediated rollback (6.915 s) and artifact substitution with rollout (3.535 s); the remaining six scenarios were observed within 0.265 s. These differences are consistent with the real control loops traversed by each injection, but one observation cannot support causal or distributional claims.

The live run materially strengthens the evidence boundary of the paper. It shows that the deficit classes are not artifacts of a Python world alone: manual mutation, exception expiry, policy supersession, artifact substitution, IAM expansion, unapproved rollback, out-of-band exposure change, approval mismatch, and approval-record loss can all be represented and decided through real Kubernetes/GitOps interfaces. It does *not* show how frequently they arise in production, how often a detector will miss them under loss or load, or whether these latencies generalize.

D. Full Protocol and Refutation Conditions

The released protocol retains the stronger follow-on design: repeat each scenario at least twenty times with randomized phase, run benign runtime and governance churn, measure drift-free false alarms, report latency distributions at multiple evaluator cadences, and separate time-to-detection from time-to-evidence. Standard experimentation guidance applies [27].

The bounded pass supports the feasibility part of **LH1**: the expected tier/class shape appeared for every scenario. It cannot establish reliable transfer, and any repeated scenario detected by T0n despite requiring a higher tier would refute the dependency analysis of Proposition 2. **LH2** predicts cadence-bounded visibility for event-carried classes; strong repeatable scenario residuals after controlling for cadence would refute it. **LH3** predicts that normalization suppresses false alarms under real churn without losing S1. **LH4** predicts a material time-to-evidence gap where approval records reside in platform-bound stores. LH2–LH4 remain open because this bounded run did not execute the repeated, churn-controlled design.

VIII. METRICS: PROPOSAL AND CRITICAL EVALUATION

We evaluate six candidate metrics against grounding in the model, operationalizability, and incentive safety, committing in advance to rejecting what cannot be justified.

Metric M1 (Governance Drift Distance, GDD)*: Proposed as: a scalar distance between approved and operational state.* Verdict: rejected as primitive; adopted as vector. *By Section IV-C, the primitive object is the component vector* $\mathsf{GD}(t)$*; a scalar GDD is legitimate only as a published-weight composite over it, with the weights understood as policy, and never as a substitute for the class-resolved report. (This mirrors the model's rejection of a scalar distance and is the suite's structural decision.)*

Metric M2 (Drift Detection Latency, DDL)*: Per class and tier: detection time minus drift onset—in events for closed-world evaluation (measured in Section VI: 0–1 events at the correct tier; 15.5 for naive comparison on S1), in wall-clock for the laboratory.* Verdict: adopted. *Onset must be defined per class (e.g., expiry instant for exception drift); the definition ships with the metric.*

Metric M3 (Unapproved Change Ratio, UCR)*: Fraction of observed change events (configuration and environment) not covered by a valid authorization basis at the time of effect.* Verdict: adopted, with an evidence caveat*: UCR is computable only where approvals are recorded durably; where they are not, UCR's denominator is measurable but its numerator degrades to undecidable, and the metric must report the undecidable mass separately* (d_{evd}) *rather than folding it into either pole.*

Metric M4 (Policy-Version Divergence, PVD)*: For each running deployment: the set (and count) of policy controls, in the currently governing version, that the deployment violates while remaining covered only by an older pinned basis—estate-wise, the distribution of "policy age" (versions or time between pinned and current).* Verdict: adopted as a set/age pair; rejected as a bare count*, since control violations are policy-weighted and a count of one critical control must not compare equal to one cosmetic control.*

Metric M5 (Authorization Drift Ratio, ADR)*: Fraction of active authorization instruments (approvals, exceptions) in an estate that are expired, revoked, or subject-mismatched while still relied upon—S2/S8-type conditions as a standing estate measurement.* Verdict: adopted*; it is directly computable from T2/T3 evidence and is the metric that makes "exception hygiene" a number.*

Metric M6 (Governance Conformance Coverage, GCC)*: Fraction of running deployments for which governance consistency is* evaluable at all—*i.e., an approved-state snapshot exists, and every component distance can be computed from available streams.* Verdict: adopted, and placed first in reporting order*: coverage bounds the meaning of every other metric (a superb ADR over 20% GCC is a statement about 20% of the estate), and low GCC is itself the actionable finding that approved-state recording is missing.*

Reporting discipline: GCC first; then the class-resolved vector metrics (DDL per tier, UCR with undecidable mass, PVD as set/age, ADR); composites last, weights published. The gaming surfaces are the usual ones—weakening policy lowers PVD; widening approvals lowers UCR—and, as in any governance measurement, the counters are meaningful only jointly with the policy and approval baselines they are computed against.

IX. Related Work

A. The Missing Join

The closest systems do observe many of the ingredients of governance drift, but they organize them around plane-local questions: desired versus observed configuration, state versus current policy, artifact versus admission rule, or environment versus benchmark. Table IV makes the gap explicit. The table compares each line of work by its *native decision object*, not by every integration a commercial product might expose. A triangle means that the line carries a useful slice but does not natively join that slice to the time-indexed admitted basis.

This fragmentation explains why the aggregate relation remained easy to miss: a deployment can be green in each available local view while the join among present state, current context, and the recorded admitted basis is broken. The contribution is not the claim that no existing mechanism can be composed to cover several columns. It is the explicit temporal three-way relation, the class-resolved verdicts it yields, and the evidence coverage required to decide it.

B. Configuration Drift and Desired-State Systems

Configuration drift detection is mature practice: reconciler status in GitOps [3, 4, 1, 2], cloud configuration recorders [6], IaC drift tools [7], and the configuration-management control tradition [14, 15] that institutionalizes baseline comparison. IaC research studies the quality and smells of the configuration artifacts themselves [29, 30, 31]. All of it operates in the configuration vocabulary and against the *current* desired state; Proposition 1 bounds what that vocabulary can decide, and Finding F4 measures the bound. Our claim is containment: this literature solves one class of six, and solves it well—T0n in our architecture *is* this literature.

C. Policy-as-Code, Admission, and Continuous Compliance

Policy engines [9, 10] and admission control [8] evaluate governance predicates at a point in time; posture management [13] and hardening baselines [28] scan environments against benchmark policy; continuous-validation features in attestation-gated deployment [11] re-check the artifact-policy slice after admission. These supply, respectively, our T1 evaluation semantics, part of T4's observation, and a one-tier precedent for continuous re-evaluation. What none provides is the *approved-state reference*: they compare state against current policy, not against the basis under which the state was admitted, so they cannot distinguish "was never compliant" from "was admitted legitimately and the world moved"—the distinction carrying the remediation semantics of Section IV-C, and the defining relation of governance drift.

D. Supply-Chain Integrity and Authorization Evidence

Attestation frameworks bind artifacts to build and review facts [19, 20, 23]; they are T3's substrate and make S4/S8-class drift decidable. Their orientation is admission-time verification; governance drift adds the post-admission temporal dimension (validity windows, revocation, supersession) and the join against G_{app}. Records-side, durable approval evidence and transparency services [32, 33, 24] determine whether the evaluator's verdicts are decidable at all—our d_{evd} component and GCC metric make that dependence explicit rather than assumed.

E. Access Control, Runtime Verification, and Drift Theory

Temporal validity of authorization has deep roots in access-control theory [17, 34] and zero-trust's continuous-evaluation stance [35]; governance drift operationalizes "continuous" for the deployment-governance plane. Runtime verification [36, 37] monitors executions against temporal specifications; our evaluator is kin—governance consistency is a safety-style property over joined state and context streams—but the specification here is not a given formula: it is reconstructed per deployment from the approved snapshot, which is why recording G_{app} is the architecture's prerequisite rather than an optimization. Concept drift [21] shares only the word: it names distribution change in data streams, not divergence from an approval basis; we flag it to prevent terminological import of its detectors, which answer a statistical question ours does not

TABLE IV
REPRESENTATIVE NATIVE DECISION SCOPE OF ADJACENT WORK. ✓ = NATIVE DECISION OBJECT; △ = PARTIAL INPUT OR SLICE; – = NOT NATIVE. THE COMPARISON IS NOT AN EXHAUSTIVE PRODUCT FEATURE AUDIT.

Line of work	Config.	Policy	Auth.	Intent	Evidence	Environ.	Joins admitted basis
Desired-state/GitOps reconciliation [3, 4, 1]	✓	–	–	–	–	–	–
IaC drift/configuration recorders [6, 7]	✓	–	–	–	–	△	–
Policy-as-code/background scans [9, 10]	–	✓	–	–	–	–	–
Posture and hardening scans [13, 28]	△	✓	–	–	–	✓	–
Supply-chain attestations [19, 23]	–	△	△	△	✓	–	–
Continuous artifact validation [11]	–	✓	✓	–	△	–	–
Governance drift (this work)	✓	✓	✓	✓	✓	✓	✓

ask. Industry usage of "governance" alongside drift management [22] treats governance as the program that suppresses configuration drift; we are aware of no prior formal treatment of drift *of* the governance relation itself, and a term search (queries "`governance drift`" alone and with `cloud`, `infrastructure`, `configuration`; ACM DL, IEEE Xplore, general scholarly indices; August 2026) found the compound used in internet-governance and organizational senses unrelated to this object.

F. Novelty, Answered Directly

Is this just configuration drift? No—with the argument's weight placed where it belongs. The load-bearing evidence is analytic: constructed, tool-realizable counterexamples in which configuration equality and governance inconsistency coexist (Figure 3), and the dependency result that no configuration-pair detector decides the property (Proposition 1). The executed staircase (Table II) supports these as an implementation-validation and noise study, not as independent field evidence. And we concede the mechanical point a skeptic will press: every governance-tier detector is itself a comparison—against a recorded basis, a validity clock, or a coverage set. The claim is not a new mechanism but a different *relation*: three-place (state, current context, admitted basis) where configuration drift is two-place and memoryless, with the consequences Section X enumerates. *Is it policy compliance scanning?* Scanning evaluates current state against current policy; governance drift is three-way—state, current context, *and approved basis*—and the third term changes both the verdicts (legitimately-admitted-since-superseded $\neq$ never-compliant) and the remediations. *Is it continuous compliance scanning or continuous validation?* Per tier, largely yes—and we credit it: Kyverno-style background scanning continuously re-evaluates resources against current policy (T1); configuration recorders and posture management watch unmanaged resources (much of T4); continuous validation re-checks artifact policy (part of T3) [10, 6, 13, 11]. Six of our nine scenarios trigger *some* existing detector. What no deployed mechanism provides is the join against the admitted basis with validity clocks and class-resolved verdicts—the increment that turns "a scanner fired" into "this running state is no longer covered by what admitted it, for this reason, with this remediation family." That increment is architectural and, we argue, decision-relevant; quantifying its operational value on real estates is exactly what the laboratory and its LH1–LH4 exist to test.

X. DISCUSSION AND LIMITATIONS

A. The Reviewer's Objection, Taken Seriously

"This is just configuration drift with more inputs." The objection deserves a precise answer beyond Section IX's. Configuration drift is a *two-place, memoryless* relation between present states; governance drift is a *three-place, history-anchored* relation among present state, evolving context, and a recorded past event. The difference is not input count but relation type. Mechanically, we concede, every detector at every tier is a comparison—monitoring always is; the question is what is compared with what. Configuration drift compares two present configurations; governance drift compares present state and present context against a *recorded past event*—the admitted basis—under validity clocks. That relational difference has observable consequences the study exhibits: (i) a detector of the first relation can be perfectly correct and carry zero information about the second (Finding F4); (ii) the second relation can change value with *no event on either configuration* (policy supersession, window expiry)—nothing a comparison could ever register, however frequently it runs; and (iii) the first relation's repair action (reconcile) can be the very mechanism that consummates a violation of the second (S6: convergence to unapproved content). A phenomenon whose detectors, event sources, and remediations are all disjoint from configuration drift's is a different research object; what they share is the word "drift" and the ambient system.

B. Legitimacy, Blame, and the Policy-Drift Question

Measuring divergence is not assigning fault. Policy drift in particular is usually the *organization's own* motion; flagging a deployment as governance-inconsistent under π_8 says migration or re-approval is due, not that anyone erred. We consider this a feature of the vector semantics: because classes are separated, an estate can adopt distinct dispositions per class (auto-reconcile configuration drift; ticket policy drift with a migration SLA; page on authorization drift; block on intent drift)—consistent with the evidence that indiscriminate heavyweight gating harms delivery without improving stability [38, 39]—turning the taxonomy into an operating policy rather than an alarm firehose. What the model deliberately does not decide—and no measurement can—is the disposition itself. Table I makes the distinction systematic: class selects a primary risk channel and response family, while blast radius, duration, reversibility, consequence, and evidence coverage determine ac-

tual severity. The vector must not be collapsed into a universal danger ranking.

C. Limitations

The approved-state prerequisite. Everything downstream of Section V assumes G_{app} was recorded durably; where it was not, verdicts degrade to undecidable, and the honest headline is GCC, not drift rates. This is a real adoption constraint, not a footnote: estates without durable approval evidence cannot measure their governance drift, and discovering that is arguably the measurement's first value. *Bounded live empirics.* The laboratory executed each scenario once on one Kind/Flux/Kyverno stack. Its 9/9 class agreement establishes feasibility and occurrence under controlled injections, but cannot estimate reliability, noise, prevalence, tail latency, stream loss, or production transfer. The closed-world staircase remains exact by construction, and the full repeated, churn-controlled protocol in Section VII is required to test how much of that shape survives real ambiguity. *Single-deployment granularity.* The model is per-deployment; estate-level aggregation (fleet drift, correlated policy supersessions) raises weighting questions we defer to the metrics' published-composite discipline. *Environment scope.* d_{env} is bounded by what σ_0 declared; assumptions never written down cannot drift detectably—an instance of the general dependence of governance measurement on capture discipline. *Remediation dynamics.* We model detection, not the control loop that acts on it; closing the loop (auto-re-approval workflows, drift-driven migration) is future work with its own stability questions.

D. Outlook

The machinery this work presupposes—durable approval evidence, and an account of how such evidence decays—and the drift semantics defined here compose naturally: approved-state snapshots are exactly the artifacts durable approval linkage produces, and evidence drift is evidence debt manifesting inside the drift vector. We have kept this manuscript self-contained, but the research program is one: make the relationship between what organizations approve and what their systems do continuously demonstrable—at admission, over time, and after the fact.

XI. Conclusion

Reconciliation solved configuration drift so well that its green status has come to stand for a property it does not check. This paper named the difference: governance drift, the divergence of operational reality from the state, policy context, authorization basis, intent, and assumptions under which it was approved. We separated six drift classes formally; established, by constructed counterexample and by a definitional-dependency argument, that configuration equality does not imply governance consistency and that no configuration-pair detector can decide it; modeled governance distance as an irreducibly component-wise vector anchored to a recorded approval event; organized detection as cumulative evidence tiers; and checked, in an executed and fully reproducible closed-world study with class-set ground truth and benign governance churn in its control, that a definition-faithful reference implementation realizes the model's detectability staircase: configuration comparison decides exactly one of nine drift scenarios; each governance tier detects precisely the scenarios whose deciding facts it carries, earning zero false alarms against governance noise; and an unnormalized comparator alarms on 71% of benign control events. The umbrella research question is thereby answered with its conditions stated: governance drift is formally representable (the vector GD), measurable exactly where the admitted basis was recorded (coverage, GCC, bounds everything else), and detectable to the depth of the evidence tiers an estate maintains. The Kubernetes/GitOps laboratory was executed once per scenario on a pinned Kind/Flux/Kyverno stack: all nine injections began from a consistent baseline, all nine yielded the expected class-resolved verdict, and observed injection-to-verdict latency ranged from 0.137 to 8.571 s. This establishes feasibility and occurrence on one controlled live stack, while deliberately leaving prevalence, reliability, production noise, and latency distributions to the released repeated-trial protocol.

The definitions are offered for reuse; the matrix is offered as a template for evaluating any governance-drift detector; and the central lesson is offered to practice in one sentence: a system can match its manifest perfectly and still be running outside everything that made it legitimate—and whether an organization can *see* that is an architectural choice, made tier by tier, that this paper makes explicit.

References

[1] OpenGitOps Project, "GitOps principles, version 1.0.0." `https://opengitops.dev/`, 2021. CNCF. Accessed August 2026.

[2] F. Beetz and S. Harrer, "GitOps: The evolution of DevOps?," *IEEE Software*, vol. 39, no. 4, pp. 70–75, 2022.

[3] Argo Project, "Argo CD: Declarative GitOps continuous delivery for Kubernetes." `https://argo-cd.readthedocs.io/`, 2026. Accessed August 2026.

[4] Flux Project, "Flux: The GitOps family of projects." `https://fluxcd.io/`, 2026. Accessed August 2026.

[5] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, omega, and Kubernetes," *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016.

[6] Amazon Web Services, "AWS Config developer guide: Detecting drift and evaluating resources." `https://docs.aws.amazon.com/config/latest/developerguide/`, 2026. Accessed August 2026.

[7] Snyk, "driftctl: Detect, track and alert on infrastructure drift." `https://docs.driftctl.com/`, 2023. Accessed August 2026.

[8] Kubernetes Project, "Admission controllers reference; validating admission policy." https://kubernetes.io/docs/reference/access-authn-authz/, 2026. Accessed August 2026.

[9] Open Policy Agent Project, "Open policy agent documentation." https://www.openpolicyagent.org/docs/, 2026. CNCF. Accessed August 2026.

[10] Kyverno Project, "Kyverno: Kubernetes native policy management." https://kyverno.io/docs/, 2026. Accessed August 2026.

[11] Google Cloud, "Binary authorization documentation, including continuous validation." https://cloud.google.com/binary-authorization/docs, 2026. Accessed August 2026.

[12] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA, USA: O'Reilly Media, 2016.

[13] Microsoft, "Cloud security posture management (CSPM) documentation, microsoft defender for cloud." https://learn.microsoft.com/en-us/azure/defender-for-cloud/concept-cloud-security-posture-management, 2026. Accessed August 2026.

[14] A. Johnson, K. Dempsey, R. Ross, S. Gupta, and D. Bailey, "Guide for security-focused configuration management of information systems," Tech. Rep. NIST Special Publication 800-128, National Institute of Standards and Technology, 2011.

[15] Joint Task Force, "Security and privacy controls for information systems and organizations," Tech. Rep. NIST Special Publication 800-53, Revision 5, National Institute of Standards and Technology, 2020.

[16] United States Congress, "Sarbanes–oxley act of 2002." Public Law 107–204, 116 Stat. 745, 2002.

[17] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[18] AXELOS, "ITIL foundation: ITIL 4 edition." TSO (The Stationery Office), 2019.

[19] OpenSSF, "SLSA specification, version 1.2." https://slsa.dev/spec/v1.2/, 2025. Accessed August 2026.

[20] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *Proceedings of the 28th USENIX Security Symposium*, pp. 1393–1410, USENIX Association, 2019.

[21] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.

[22] ControlMonkey, "Cloud governance best practices: Five ways to prevent drift." https://controlmonkey.io/blog/cloud-governance-best-practices-devops/, 2025. Representative industry usage conflating drift management with governance; accessed August 2026.

[23] Z. Newman, J. S. Meyers, and S. Torres-Arias, "Sigstore: Software signing for everybody," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 2353–2367, ACM, 2022.

[24] Amazon Web Services, "AWS CloudTrail user guide." https://docs.aws.amazon.com/awscloudtrail/latest/userguide/, 2026. Accessed August 2026.

[25] OpenTelemetry Project, "The opentelemetry specification." https://opentelemetry.io/docs/specs/otel/, 2026. CNCF. Accessed August 2026.

[26] Kubernetes SIGs, "kind: Kubernetes in Docker." https://kind.sigs.k8s.io/, 2026. Accessed August 2026.

[27] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Germany: Springer, 2012.

[28] National Security Agency and Cybersecurity and Infrastructure Security Agency, "Kubernetes hardening guide, version 1.2." Cybersecurity Technical Report, 2022.

[29] A. Rahman, R. Mahdavi-Hezaveh, and L. Williams, "A systematic mapping study of infrastructure as code research," *Information and Software Technology*, vol. 108, pp. 65–77, 2019.

[30] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pp. 164–175, IEEE, 2019.

[31] T. Sharma, M. Fragkoulis, and D. Spinellis, "Does your configuration code smell?," in *Proceedings of the 13th International Conference on Mining Software Repositories (MSR)*, pp. 189–200, ACM, 2016.

[32] H. Birkholz, A. Delignat-Lavaud, C. Fournet, Y. Deshpande, and S. Lasker, "An architecture for trustworthy and transparent digital supply chains (SCITT)." Internet-Draft draft-ietf-scitt-architecture-22, IETF, 2025. Work in Progress, October 2025.

[33] L. Moreau and P. Missier, "PROV-DM: The PROV data model." W3C Recommendation, 2013. https://www.w3.org/TR/prov-dm/. April 30, 2013.

[34] D. D. Clark and D. R. Wilson, "A comparison of commercial and military computer security policies," in *Proceedings of the 1987 IEEE Symposium on Security and Privacy*, pp. 184–194, IEEE, 1987.

[35] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," Tech. Rep. NIST Special Publication 800-207, National Institute of Standards and Technology, 2020.

[36] M. Leucker and C. Schallhart, "A brief account of runtime verification," *Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009.

[37] A. Pnueli, "The temporal logic of programs," in *Proceedings of the 18th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 46–57, IEEE, 1977.

[38] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps*. Portland, OR, USA: IT Revolution Press, 2018.

[39] DevOps Research and Assessment (DORA), "Accelerate: State of DevOps 2019." Google Cloud, 2019. https://dora.dev/research/2019/. Accessed August 2026.